



UNIVERSITÀ DI PARMA

Le Funzioni

E Pluribus Unum

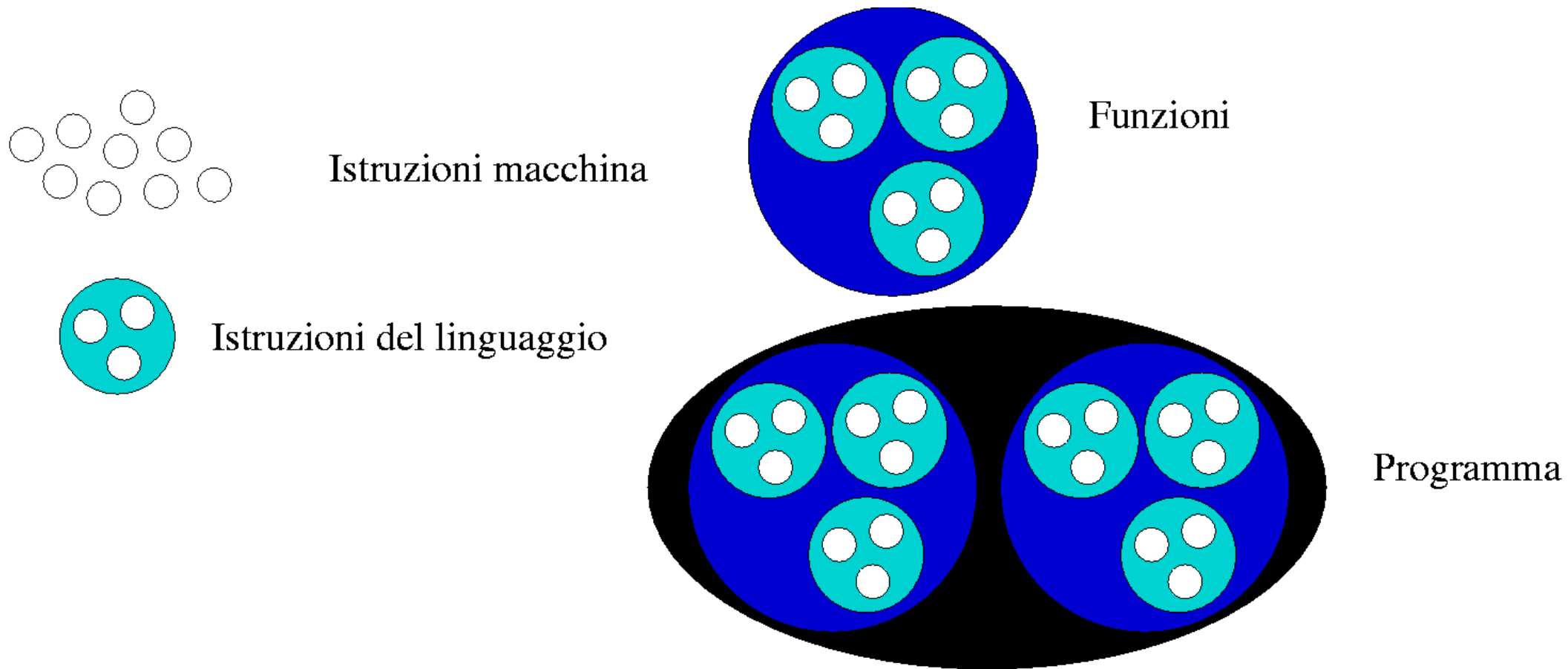
Autore Ignoto, Moretum, I sec. a.C.

- Definire una funzione
 - Tipo restituito
 - Parametri Formali
 - Passaggio dei dati
 - I prototipi
- La Ricorsione

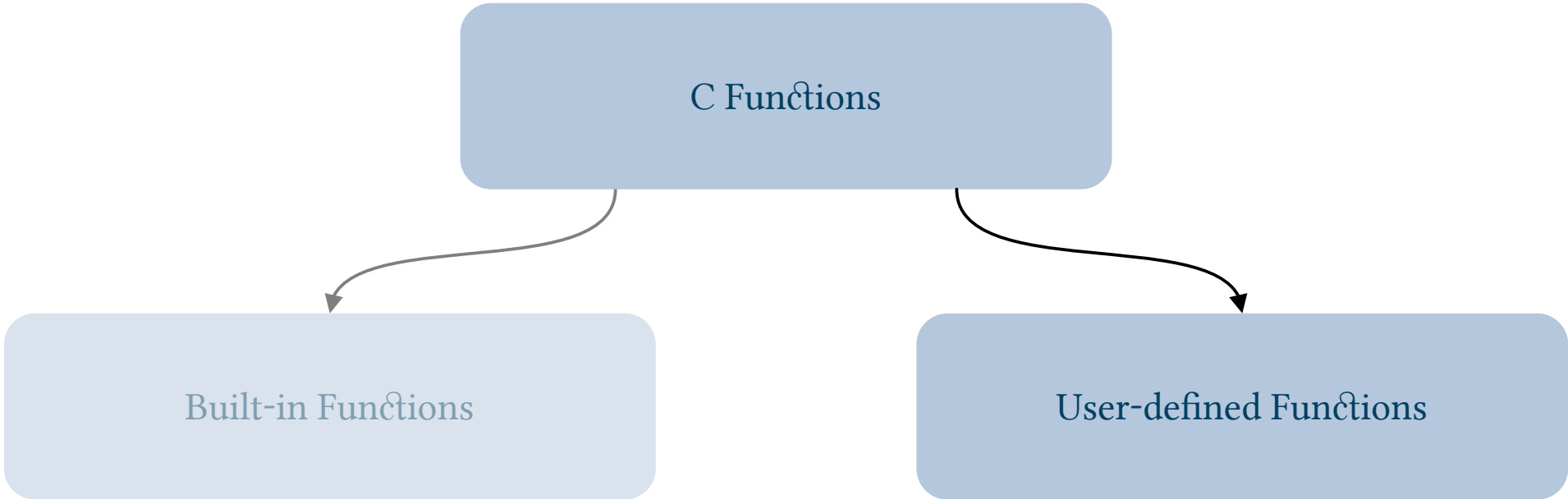
SUMMARY



We have already talked about this



C Functions



```
graph TD; A[C Functions] --> B[Built-in Functions]; A --> C[User-defined Functions];
```

Built-in Functions

```
printf()  
scanf()  
malloc()  
rand()  
...
```

User-defined Functions

Defining a new function

- Already done → main()
- We have to write our function signature + body

Returned data type
(int, float, char...)

Name of function

Formal parameters
list

`<return type>`

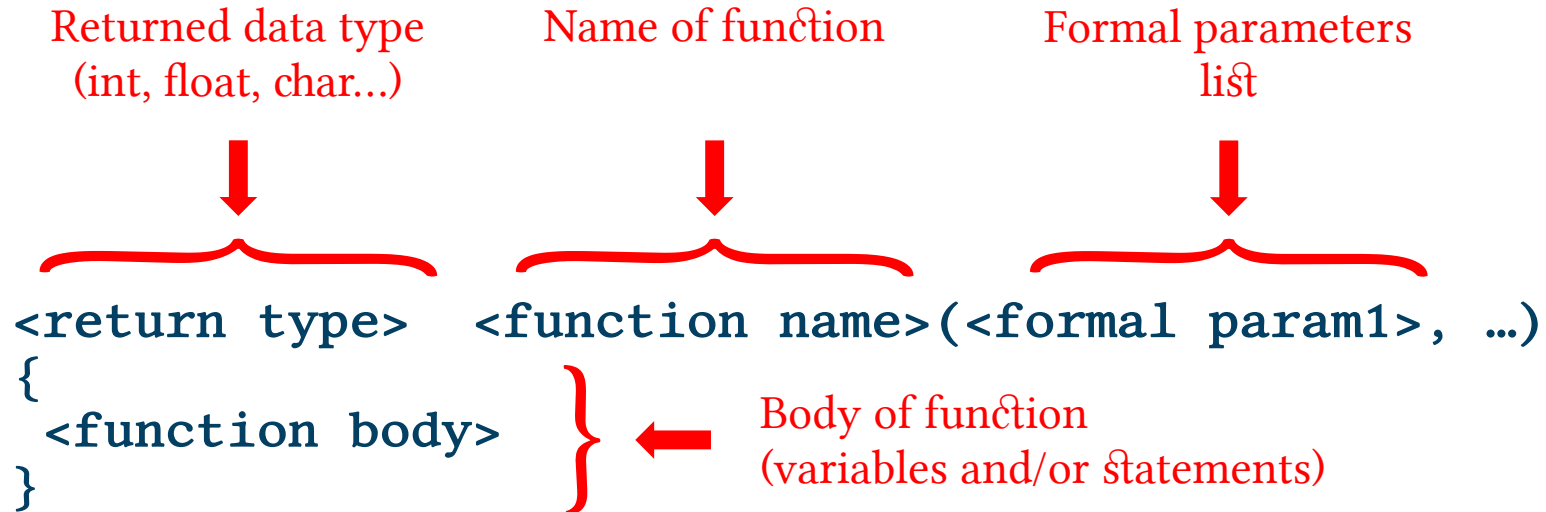
`<function name>(<formal param1>, ...)`

`{`

`<function body>`

`}`

Body of function
(variables and/or statements)



- Tipo di dato restituito
 - Quelli già visti... e quelli che vedremo!
 - Nel caso di puntatori $\rightarrow *$
 - E se non devo restituire nessun dato?
 - void
- Nome
 - Regole già viste per le variabili
 - Opportunamente mnemonico!

- Elenco parametri formali
 - Lista separata da “,”
 - Specificare tipo e nome
 - Se non necessari $\rightarrow$ void

- Corpo funzione
 - Racchiuso tra { }
 - Codice che realizza funzione
 - Può essere anche vuoto ({} still needed!)
 - *Stub*
 - Utile in fasi iniziali sviluppo
 - Se restituisco qualcosa obbligo `return`

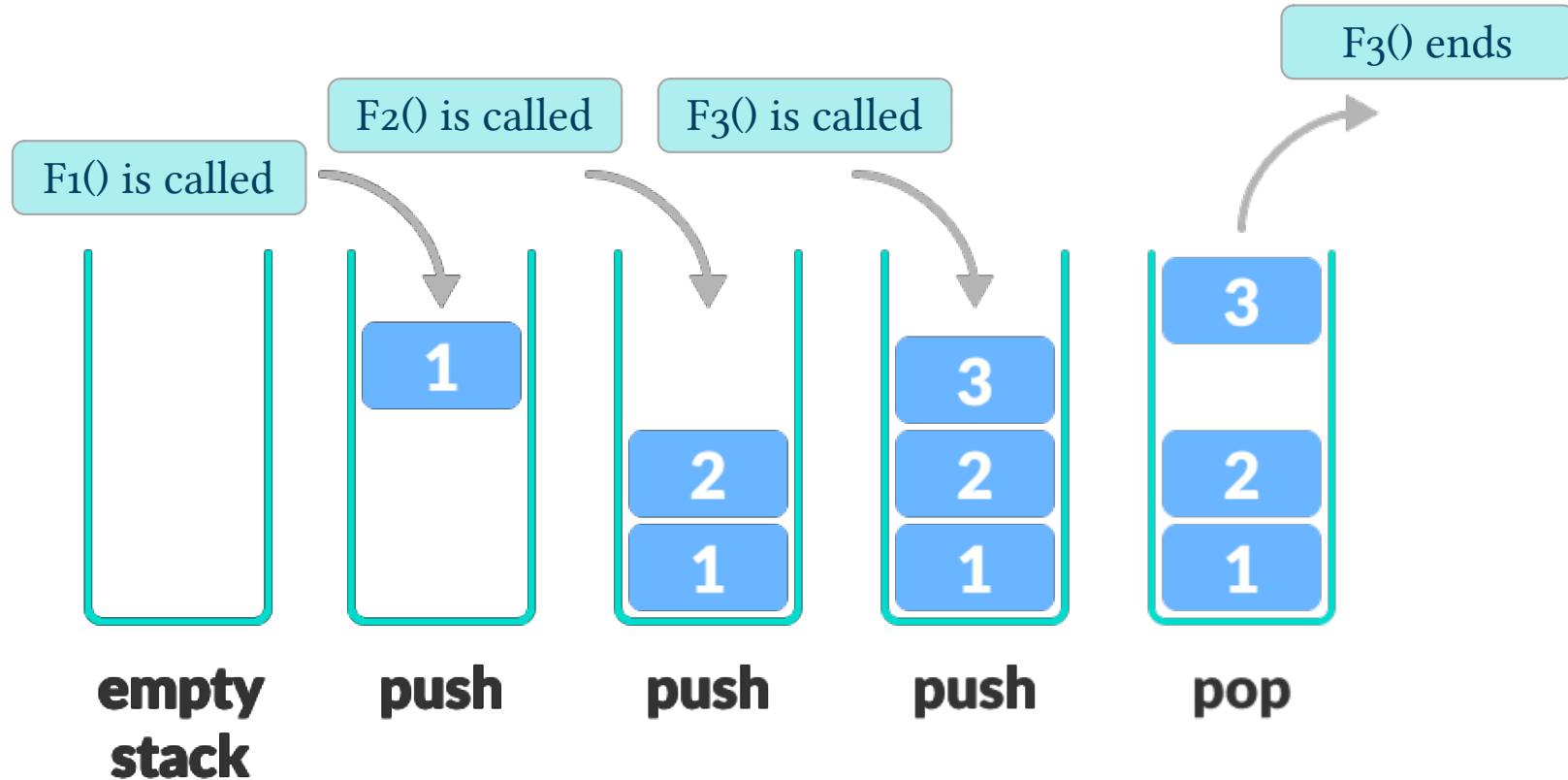
- Il compilatore quando incontra una chiamata a funzione deve sapere di che funzione si tratta
 - Il codice viene letto dall'alto in basso
 - Non sempre possibile definire una funzione prima di una sua chiamata
- Soluzione → riga di definizione della funzione (prototipo)

- L'invocazione di una funzione definita da noi segue le stesse regole già viste per le funzioni predefinite
 - Devo passare un numero e un tipo di argomenti coerenti con i parametri della funzione
 - Attenzione alle conversioni di tipo!

Invoking a function → memory

- When we invoke a function a specific memory area is involved
 - Stack
- And it contains all data needed to execute the function
- **Activation record:**
 - Parameters of the callee
 - Function arguments
 - Local variables
 - Return address of the function itself
- **LIFO: Last In First Out**

Invoking a function → memory



- Quando invoco una funzione → argomenti
 - O anche “parametri attuali” o “effettivi”
 - Valori passati ad una funzione
- Nella definizione di una funzione → parametri formali
 - Di fatto variabili locali alla funzione
 - A loro viene assegnato il valore degli argomenti

- Passaggio argomenti, due modalità
 - Per valore
 - Per indirizzo

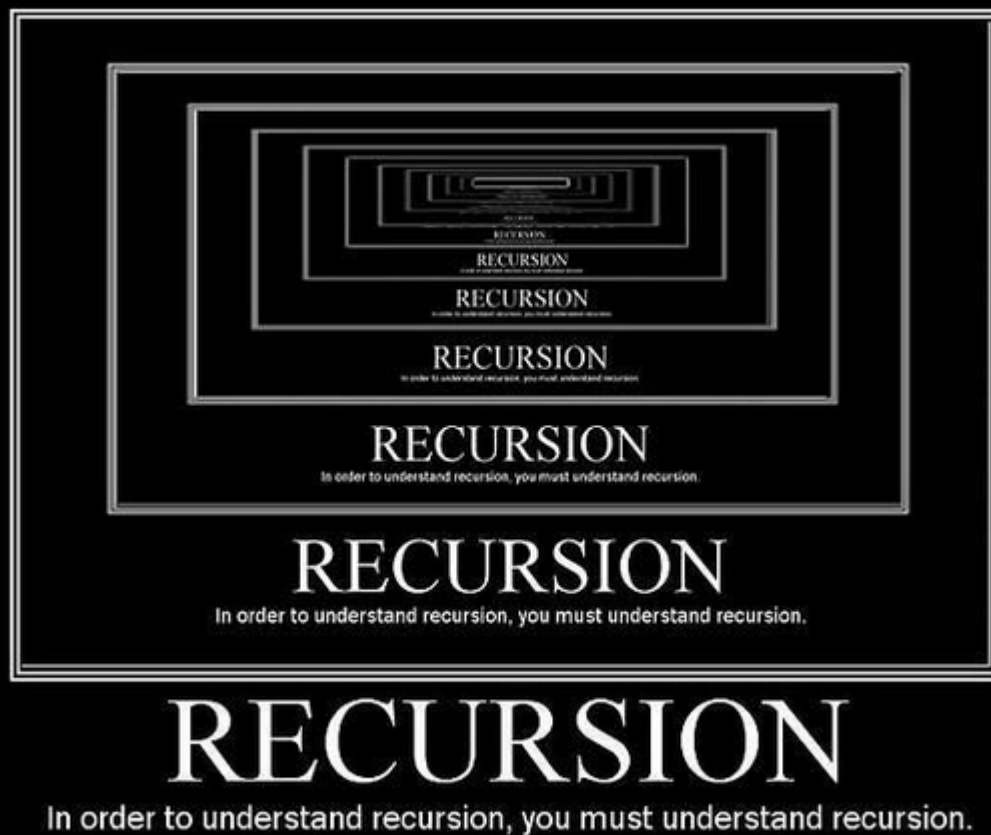
- Passaggio argomenti, per valore
 - È il default per tutti i dati ad eccezione degli array
 - Viene creata una **copia** dell'argomento
 - La funzione quindi lavora su **copie**, non sul **dato originale**
 - Impossibile agire su variabili passate alla funzione
- Esempi
 - `b = sqrt(a)` → sto passando il **contenuto** della variabile `a`
 - `b = sqrt(12)` → posso infatti anche usare direttamente un numero

- Passaggio argomenti per indirizzo
 - È il default per gli array
 - Viene passato alla funzione l'indirizzo del dato
 - Attenzione al tipo di dato!
 - La funzione può modificare il dato originale
 - Effetti collaterali
- Esempio
 - `scanf("%f", &a);` → modificherà il contenuto di a (!)

- When passing arrays we could have to deal with size
 - For strings maybe it is not necessary
- Three different approaches depending on situation
 - Fixed size array → size is fully specified
 - VLA → size is given as parameter
 - As pointer → similar to previous case

- Permette
 - Uscita funzione e
 - Restituzione valore per funzioni non void
- Non obbligatorio per funzioni void
- Può restituire uno e un solo valore

return (<espressione>);



- Approccio molto elegante e semplice che permette di risolvere problemi anche complessi
- La ricorsione si può applicare a tutti quei problemi in cui la risoluzione dipende dalla risoluzione dello stesso problema però di ordine di grandezza inferiore ovvero più semplice
- Esempio: il fattoriale

$$n! = n \cdot (n - 1)!$$

- Esempio: fattoriale
 - Il fattoriale di n lo posso calcolare come funzione del fattoriale di $n-1$
 - Esistono inoltre casi specifici

$$n! = \begin{cases} 1 & \text{per } n=0 \\ n \cdot (n-1)! & \text{per } n>1 \end{cases}$$

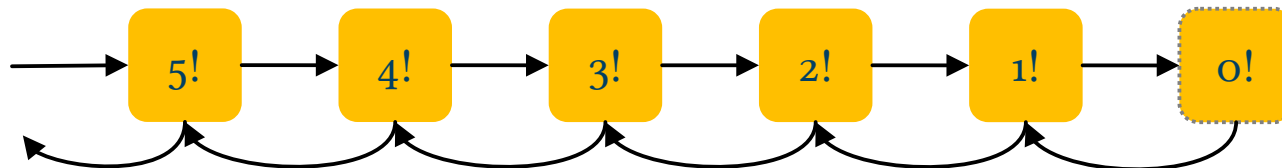
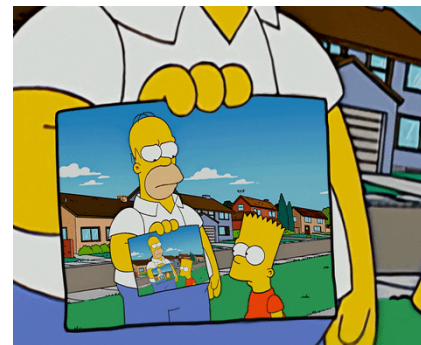
- Una soluzione ricorsiva di un problema è facilmente trasponibile in codice
- **Funzione che chiama se stessa!**

$$n! = \begin{cases} 1 & \text{per } n=0 \\ n \cdot (n-1)! & \text{per } n \geq 1 \end{cases}$$

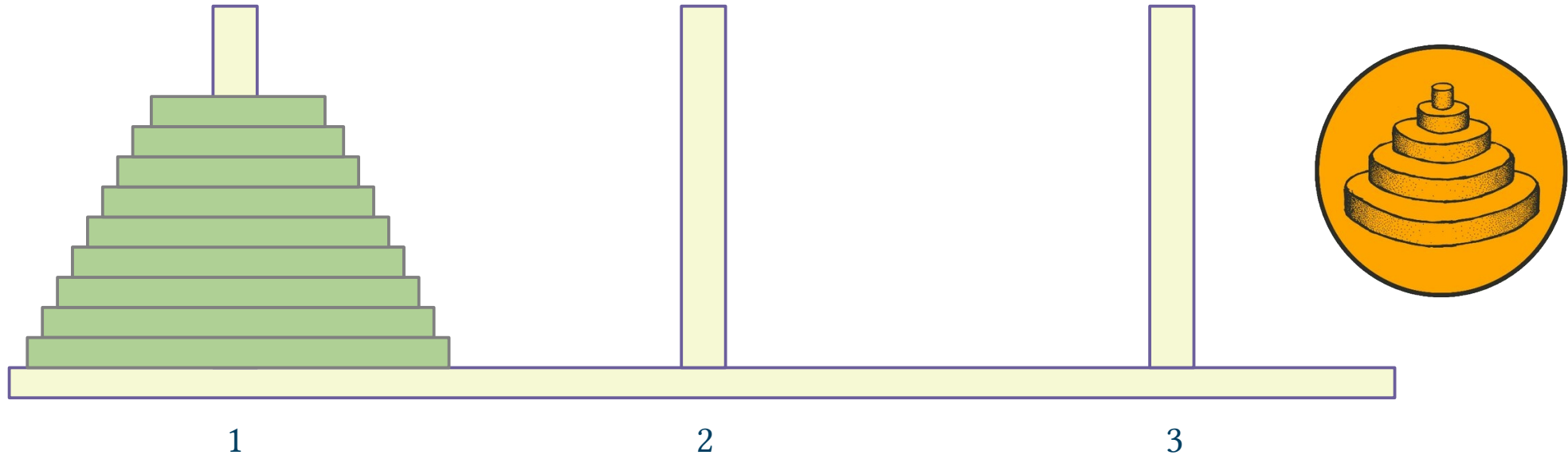


```
unsigned long long fact(unsigned long long n)
{
    if(n==0) return 1;
    return n*fact(n-1);
}
```

- Chiamate ricorsive implicano avere tante “istanze” della stessa funzione
 - Ognuna con le sue variabili locali



- Torre di Hanoi



Obiettivo: spostare n dischi dal piolo 1 al piolo 3

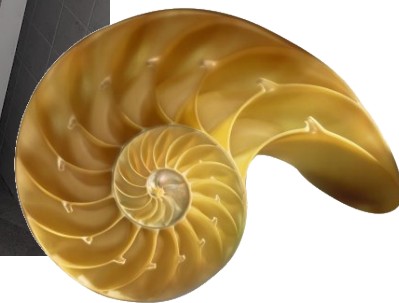
- Torre di Hanoi, definiamo
 - $T(n,x,y) \rightarrow$ sposta n dischi da piolo x a piolo y
 - $D(x,y) \rightarrow$ sposta un singolo disco da piolo x a piolo y

$$T(n,1,3) = \begin{cases} \text{per } n=0 & \text{niente} \\ \text{per } n>1 & T(n-1,1,2) + D(1,3) + T(n-1,2,3) \end{cases}$$

- Errori Comuni
 - Mancanza condizione di uscita
 - La semplicità può mascherare inefficienza



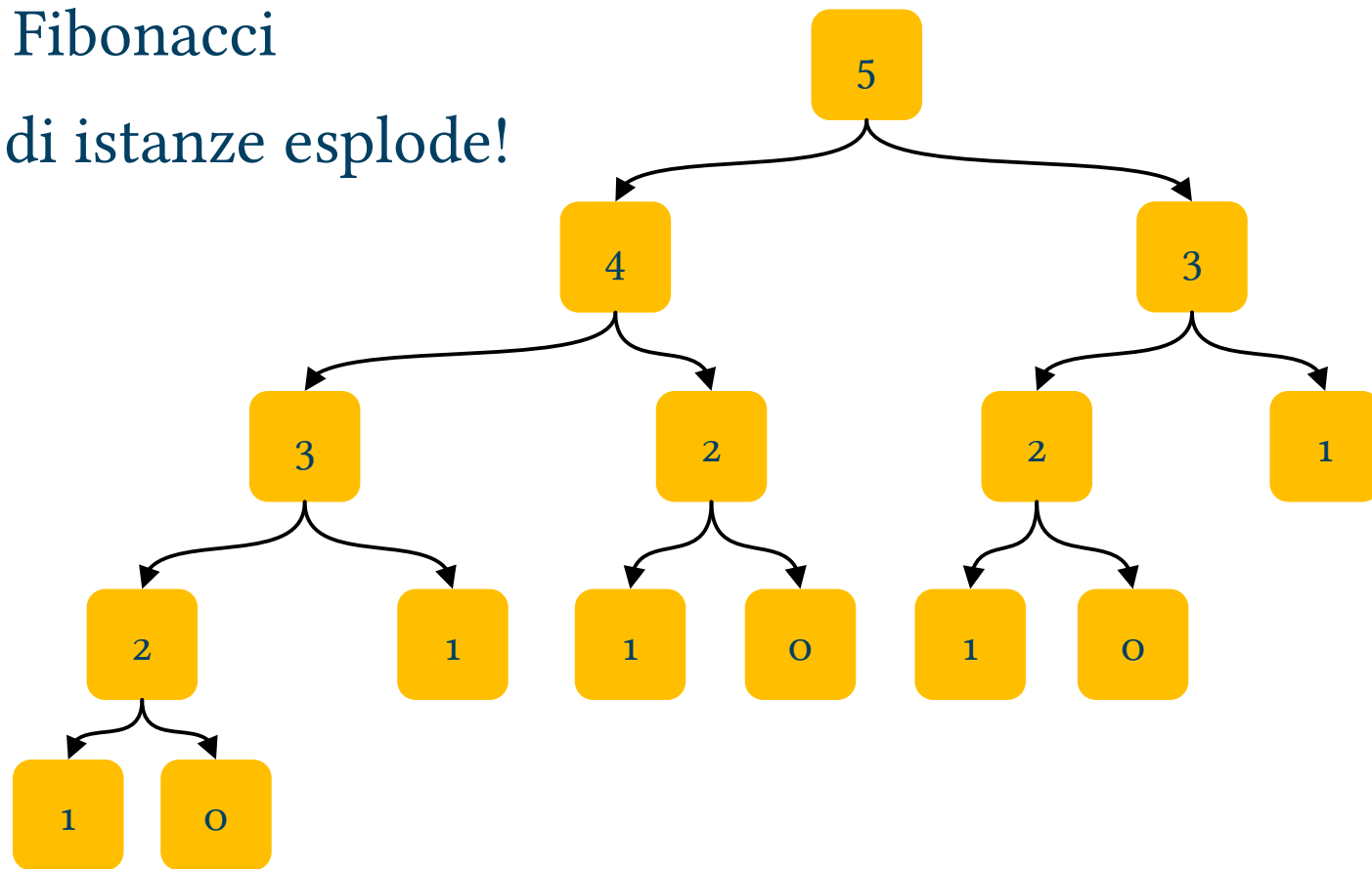
- Numeri di Fibonacci
 - 0, 1, 1, 2, 3, 5, 8, 13, 21,...



$$f(n) = \begin{cases} \text{per } n=0 & 0 \\ \text{per } n=1 & 1 \\ \text{per } n>1 & f(n-1) + f(n-2) \end{cases}$$


```
unsigned long fibo(unsigned long n){  
    if(n==0) return 0;  
    if(n==1) return 1;  
  
    return fibo(n-1)+fibo(n-2);  
}
```

- Numeri di Fibonacci
- Il numero di istanze esplode!



- L'implementazione ricorsiva vista in precedenza non è efficiente
- Si può dimostrare che se S è il tempo necessario al calcolo del numero di fibonacci di indice $n \rightarrow f(n)$, il calcolo di $f(n+1)$ richiede circa $1,6 \times S$
- Ad esempio se il calcolo di $f(n)$ richiede 1 s, il calcolo di $f(n+18)$ richiederà un'ora
- Soluzione $\rightarrow$ dynamic programming (vedi esempio)

- Entrambe sono di fatto ripetizioni
 - Ciclo esplicito vs Ciclo implicito
 - Tutto ciò che posso risolvere con la ricorsione è risolvibile iterativamente
 - Entrambe vogliono condizione di uscita
 - Rischio ciclo infinito
- Ricorsione → codice più semplice
 - Spesso più facile e codice più compatto
 - Contro: overhead chiamate di funzione



UNIVERSITÀ DI PARMA

Le Funzioni



KEEP
CALM
IT'S
QUESTION
TIME

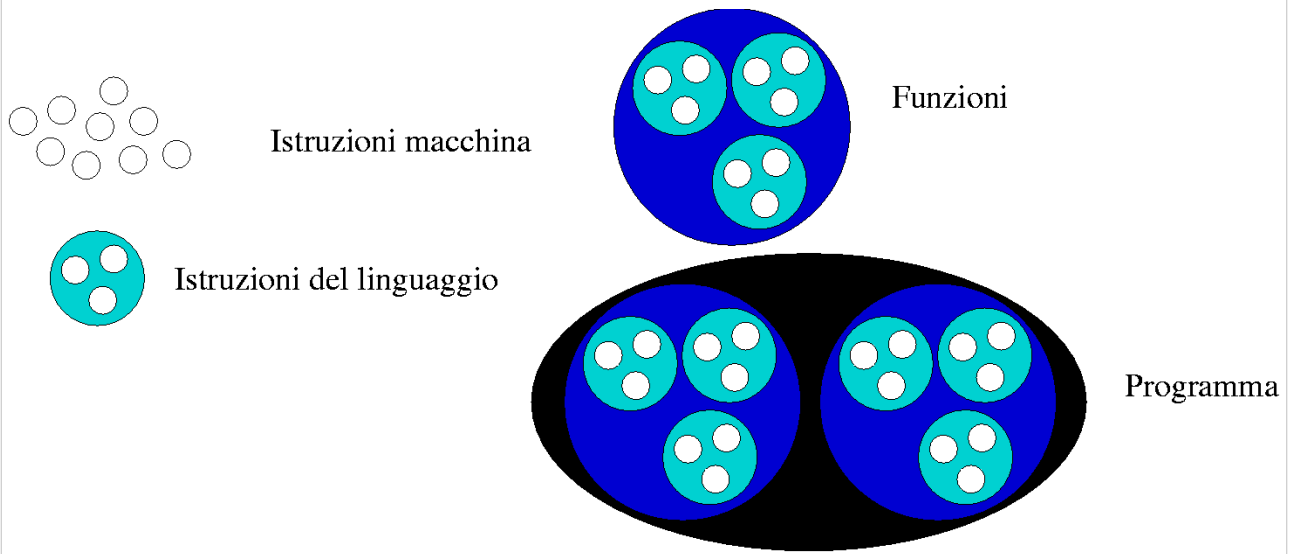
E Pluribus Unum

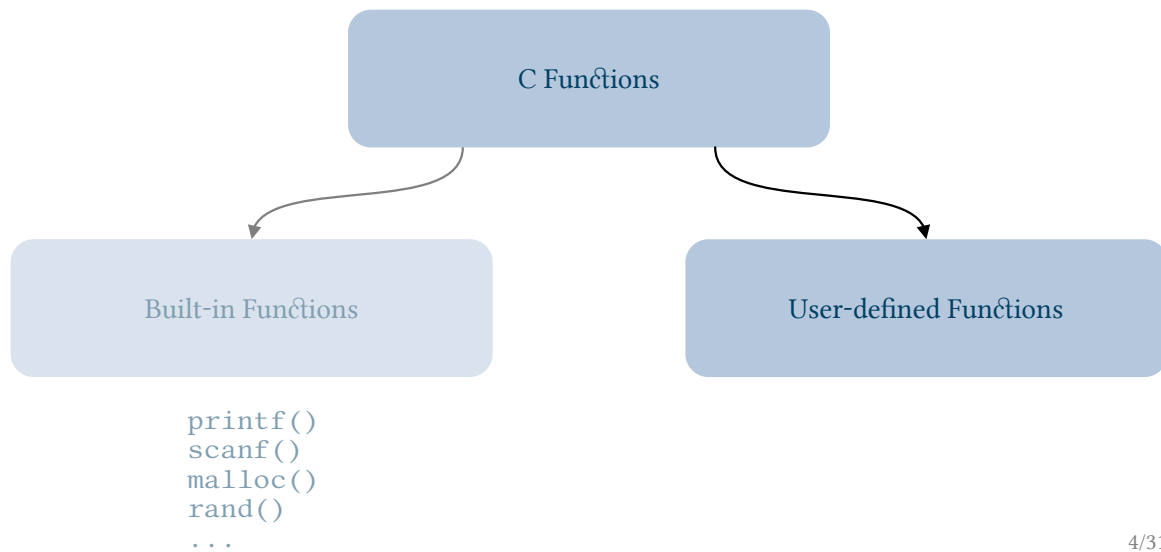
Autore Ignoto, Moretum, I sec. a.C.

- Definire una funzione
 - Tipo restituito
 - Parametri Formali
 - Passaggio dei dati
 - I prototipi
- La Ricorsione



We have already talked about this





Defining a new function

- Already done → `main()`
- We have to write our function signature + body

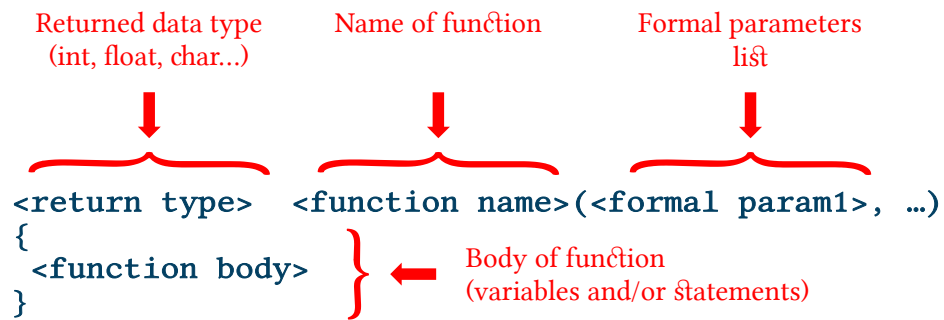
Returned data type
(int, float, char...)

Name of function

Formal parameters
list

`<return type> <function name>(<formal param1>, ...)`
`{`
`<function body>`
`}`

Body of function
(variables and/or statements)



- Tipo di dato restituito
 - Quelli già visti... e quelli che vedremo!
 - Nel caso di puntatori $\rightarrow *$
 - E se non devo restituire nessun dato?
 - void
- Nome
 - Regole già viste per le variabili
 - Opportunamente mnemonico!

- Elenco parametri formali
 - Lista separata da “,”
 - Specificare tipo e nome
 - Se non necessari → void

- Corpo funzione
 - Racchiuso tra {}
 - Codice che realizza funzione
 - Può essere anche vuoto ({} still needed!)
 - *Stub*
 - Utile in fasi iniziali sviluppo
 - Se restituisco qualcosa obbligo `return`

- Il compilatore quando incontra una chiamata a funzione deve sapere di che funzione si tratta
 - Il codice viene letto dall'alto in basso
 - Non sempre possibile definire una funzione prima di una sua chiamata
- Soluzione → riga di definizione della funzione (prototipo)

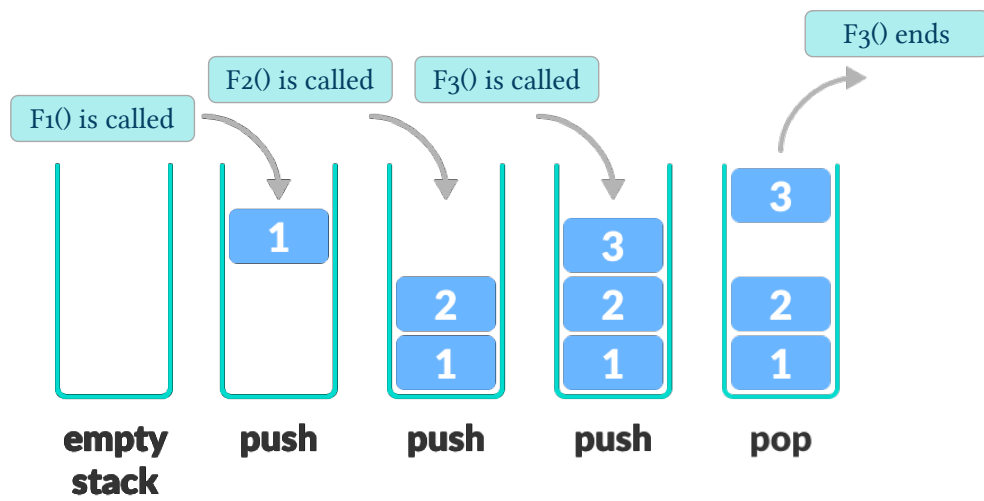


- L'invocazione di una funzione definita da noi segue le stesse regole già viste per le funzioni predefinite
 - Devo passare un numero e un tipo di argomenti coerenti con i parametri della funzione
 - Attenzione alle conversioni di tipo!



- When we invoke a function a specific memory area is involved
 - Stack
- And it contains all data needed to execute the function
- **Activation record:**
 - Parameters of the callee
 - Function arguments
 - Local variables
 - Return address of the function itself
- **LIFO: Last In First Out**

Invoking a function → memory





- Quando invoco una funzione → argomenti
 - O anche “parametri attuali” o “effettivi”
 - Valori passati ad una funzione
- Nella definizione di una funzione → parametri formali
 - Di fatto variabili locali alla funzione
 - A loro viene assegnato il valore degli argomenti

- Passaggio argomenti, due modalità
 - Per valore
 - Per indirizzo



- Passaggio argomenti, per valore
 - È il default per tutti i dati ad eccezione degli array
 - Viene creata una **copia** dell'argomento
 - La funzione quindi lavora su **copie**, non sul **dato originale**
 - Impossibile agire su variabili passate alla funzione
- Esempi
 - `b = sqrt(a)` → sto passando il **contenuto** della variabile `a`
 - `b = sqrt(12)` → posso infatti anche usare direttamente un numero



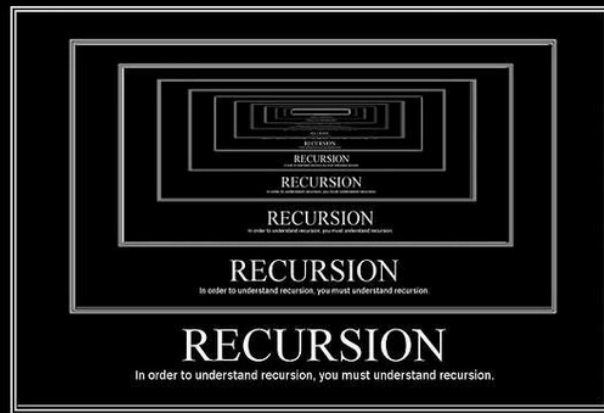
- Passaggio argomenti per indirizzo
 - È il default per gli array
 - Viene passato alla funzione l'indirizzo del dato
 - Attenzione al tipo di dato!
 - La funzione può modificare il dato originale
 - Effetti collaterali
- Esempio
 - `scanf("%f", &a);` → modificherà il contenuto di a (!)



- When passing arrays we could have to deal with size
 - For strings maybe it is not necessary
- Three different approaches depending on situation
 - Fixed size array → size is fully specified
 - VLA → size is given as parameter
 - As pointer → similar to previous case

- Permette
 - Uscita funzione e
 - Restituzione valore per funzioni non void
- Non obbligatorio per funzioni void
- Può restituire uno e un solo valore

return (<espressione>);



RECURSION
In order to understand recursion, you must understand recursion.

- Approccio molto elegante e semplice che permette di risolvere problemi anche complessi
- La ricorsione si può applicare a tutti quei problemi in cui la risoluzione dipende dalla risoluzione dello stesso problema però di ordine di grandezza inferiore ovvero più semplice
- Esempio: il fattoriale

$$n! = n \cdot (n-1)!$$

- Esempio: fattoriale
 - Il fattoriale di n lo posso calcolare come funzione del fattoriale di $n-1$
 - Esistono inoltre casi specifici

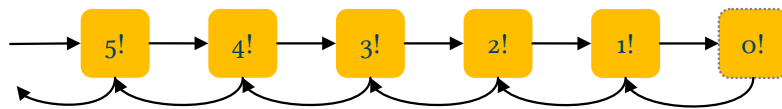
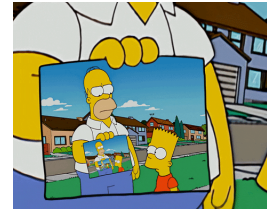
$$n! = \begin{cases} 1 & \text{per } n=0 \\ n \cdot (n-1)! & \text{per } n>1 \end{cases}$$

- Una soluzione ricorsiva di un problema è facilmente trasponibile in codice
- **Funzione che chiama se stessa!**

		<pre>unsigned long long fact(unsigned long long n) { if(n==0) return 1; return n*fact(n-1); }</pre>
$n! = \begin{cases} 1 & \text{per } n=0 \\ n \cdot (n-1)! & \text{per } n \geq 1 \end{cases}$		

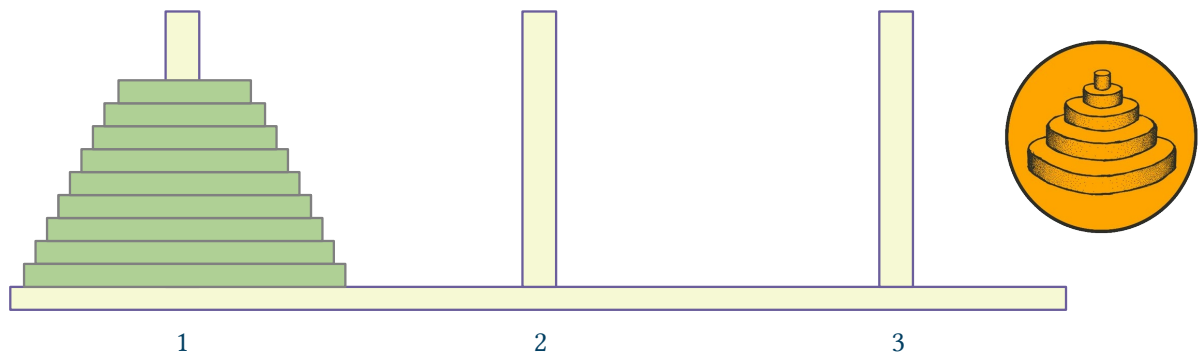


- Chiamate ricorsive implicano avere tante “istanze” della stessa funzione
 - Ognuna con le sue variabili locali





- Torre di Hanoi



Obbiettivo: spostare n dischi dal piolo 1 al piolo 3



- Torre di Hanoi, definiamo
 - $T(n,x,y) \rightarrow$ sposta n dischi da piolo x a piolo y
 - $D(x,y) \rightarrow$ sposta un singolo disco da piolo x a piolo y

$$T(n,1,3) = \begin{cases} \text{per } n=0 & \text{niente} \\ \text{per } n>1 & T(n-1,1,2) + D(1,3) + T(n-1,2,3) \end{cases}$$

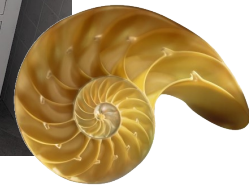


- Errori Comuni
 - Mancanza condizione di uscita
 - La semplicità può mascherare inefficienza





- Numeri di Fibonacci
 - 0, 1, 1, 2, 3, 5, 8, 13, 21,...



$$f(n) = \begin{cases} \text{per } n=0 & 0 \\ \text{per } n=1 & 1 \\ \text{per } n>1 & f(n-1) + f(n-2) \end{cases}$$



```
unsigned long fibo(unsigned long n){  
    if(n==0) return 0;  
    if(n==1) return 1;  
    return fibo(n-1)+fibo(n-2);  
}
```

- Fibonacci
di istanze esplode!
-
- ```
graph TD; 5[5] --> 4[4]; 5 --> 3[3]; 4 --> 3_1[3]; 4 --> 2_1[2]; 3_1 --> 2_1_1[2]; 3_1 --> 1_1[1]; 2_1_1 --> 1_1_1[1]; 2_1_1 --> 0_1_1[0]; 2_1 --> 1_1_2[1]; 2_1 --> 0_1_2[0]; 3 --> 2_2[2]; 3 --> 1_2[1]; 2_2 --> 1_2_1[1]; 2_2 --> 0_2_1[0]; 1_2 --> 1_2_2[1];
```
- The diagram illustrates a recursion tree for the 5th Fibonacci number. The root node is 5, which branches into 4 and 3. Node 4 branches into 3 and 2, while node 3 branches into 2 and 1. The tree continues to branch down to base cases of 1 and 0, showing a total of 15 nodes and 14 edges, which represents the exponential growth of recursive calls.



- L'implementazione ricorsiva vista in precedenza non è efficiente
- Si può dimostrare che se  $S$  è il tempo necessario al calcolo del numero di fibonacci di indice  $n \rightarrow f(n)$ , il calcolo di  $f(n+1)$  richiede circa  $1,6 \times S$
- Ad esempio se il calcolo di  $f(n)$  richiede 1 s, il calcolo di  $f(n+18)$  richiederà un'ora
- Soluzione  $\rightarrow$  dynamic programming (vedi esempio)



- Entrambe sono di fatto ripetizioni
  - Ciclo esplicito vs Ciclo implicito
  - Tutto ciò che posso risolvere con la ricorsione è risolvibile iterativamente
  - Entrambe vogliono condizione di uscita
    - Rischio ciclo infinito
- Ricorsione → codice più semplice
  - Spesso più facile e codice più compatto
  - Contro: overhead chiamate di funzione

